

INHERITANCE

- Inheritance in C++ is one of the most important concepts in object-oriented programming because it allows us to build new classes from existing ones

1. Basic Inheritance

```
class Base {  
private:  
    int Base_private;  
public:  
    Base() {  
        Base_private = 1;  
    }  
    void PrintBase() {  
        cout << "Base_private number : " << Base_private << endl;  
    }  
};
```

Class Definition

- We start with a Base class, it has a private integer Base_private that is initialized to 1 in the constructor.
- The function PrintBase() prints this value. Then we define a Derived class that inherits publicly from Base.
- It adds its own private integer Derived_private, initialized to 4, and a function PrintDerived() to print it.
- This shows that the derived class not only has its own members but also inherits the ability to use functions from the base class.

Main Function

```
int main() {  
    Derived derived;  
    derived.PrintBase();  
    derived.PrintDerived();  
    return 0;  
}
```

- When we create a Derived object, the base constructor runs first, then the derived constructor. Calling PrintBase() prints the base value, and PrintDerived() prints the derived value.

2. Access Control Specifiers

```
class Base {  
    private:  
        int Base_private;  
    protected:  
        int Base_protected;  
    public:  
        int Base_public;  
};
```

Class Definition

The Base class has three types of members:

- private → only accessible inside Base.
- protected → accessible inside Base and derived classes.
- public → accessible everywhere.

The function setBasenum() in Base can set all three because it belongs to the class itself.

In the Derived class, trying to set Base_private causes a compile error, but Base_protected and Base_public work fine.

Main Function

```
int main() {  
    Base base;  
    base.Base_public = 3; // ✅ works  
    // base.Base_private = 1; ❌ error  
    // base.Base_protected = 2; ❌ error  
    return 0;  
}
```

- In main(), we can only access Base_public directly. Attempting to access Base_private or Base_protected outside the class results in errors.
- This demonstrates how access specifiers control visibility and protect data from unwanted access.

3. Constructor in Derived Class

```
class Base {
public:
    Base() {
        cout << "Constructor of base class" << endl;
    }
    Base(int n) {
        cout << "Constructor of base class with value: " << n << endl;
    }
};

class Derived : public Base {
public:
    Derived(int n1) {
        cout << "Constructor of derived class" << endl;
    }
    Derived(int n1, int n2) : Base(n1) {
        cout << "Constructor of derived class" << endl;
    }
};
```

Class Definition

The Base class has two constructors: a default one and a parameterized one. The Derived class also has two constructors:

- One without an initializer list, which automatically calls the default base constructor.
- One with an initializer list, which explicitly calls the parameterized base constructor.

Main Function

```
int main() {
    Derived d1(2);    // calls default Base constructor
    Derived d2(1, 2); // calls parameterized Base constructor
    return 0;
}
```

- Creating Derived d1(2); calls the default base constructor followed by the derived constructor.
- Creating Derived d2(1,2); calls the parameterized base constructor followed by the derived constructor.

This shows that base constructors always run before derived constructors, ensuring proper initialization order.

4. Destructor in Inheritance

```
class Base {
public:
    ~Base() {
        cout << "Destructor of base class" << endl;
    }
};

class Derived : public Base {
public:
    ~Derived() {
        cout << "Destructor of derived class" << endl;
    }
};
```

Class Definition

- Both Base and Derived have destructors that print messages when called.

Main Function

```
int main() {
    Derived *ptr = new Derived();
    delete ptr;
    return 0;
}
```

- When we create a Derived object dynamically and delete it, the destructor of Derived runs first, then the destructor of Base.
- This order ensures that resources allocated in the derived class are cleaned up before the base class is destroyed.

5. Person–Student Example

```
class Person {
private:
    char *name;
    int age;
public:
    Person(int age, const char *name) {
        this->age = age;
        this->name = new char[strlen(name) + 1];
        strcpy(this->name, name);
    }
    ~Person() {
        delete [] name;
    }
    void PrintPerson() {
        cout << "My age is " << age << endl;
        cout << "My name is " << name << endl;
    }
};
```

Class Definition

- The Person class stores name and age. In the constructor, memory is allocated dynamically for the name using new char[]. In the destructor, memory is freed using delete[].
- The Student class inherits from Person and adds major, also managed with dynamic memory.

Main Function

```
int main() {
    Student s(20, "Gilbert", "Computer Science");
    s.PrintStudent();
    return 0;
}
```

- Creating a Student object initializes both Person and Student. Calling PrintStudent() first prints the age and name, then the major.
- This example shows how inheritance works with dynamic memory and why destructors are essential to prevent memory leaks.

6. Multiple Inheritance

```
class Mid1 {
public:
    Mid1(int num) { cout << "Mid1 constructor" << endl; }
    void Mid1_function() { cout << "Mid1 function" << endl; }
};

class Mid2 {
public:
    Mid2(int num) { cout << "Mid2 constructor" << endl; }
    void Mid2_function() { cout << "Mid2 function" << endl; }
};

class Final : public Mid1, public Mid2 {
public:
    Final(int num1, int num2) : Mid1(num1), Mid2(num2) {}
};
```

Class Definition

- Two classes, Mid1 and Mid2, each have a constructor and a function. The Final class inherits from both. Its constructor initializes both base classes.

Main Function

```
int main() {
    Final final(1, 2);
    final.Mid1_function();
    final.Mid2_function();
    return 0;
}
```

- Creating a Final object calls both base constructors. Calling Mid1_function() and Mid2_function() prints values from both bases.
- This shows how multiple inheritance allows combining features from different classes, but it can also lead to ambiguity if both bases have functions with the same name.

7. Virtual Inheritance

```
class Base {
public:
    Base(int num) { cout << "Base constructor" << endl; }
    void Base_function() { cout << "Base function" << endl; }
};

class Mid1 : virtual public Base {
public:
    Mid1(int num) : Base(num) {}
};

class Mid2 : virtual public Base {
public:
    Mid2(int num) : Base(num) {}
};

class Final : public Mid1, public Mid2 {
public:
    Final(int num) : Base(num), Mid1(num), Mid2(num) {}
};
```

Class Definition

- The Base class has a function. Mid1 and Mid2 inherit virtually from Base. Final inherits from both Mid1 and Mid2. Its constructor initializes the common Base.

Main Function

```
int main() {
    Final final(1);
    final.Base_function();
    return 0;
}
```

- Creating a Final object ensures only one copy of Base exists. Calling Base_function() works without ambiguity.